Vector-Vector

Inner Product (Dot Product)

For every $x, y \in \mathbb{R}^d$, it follows $\sum_{i=1}^{d} x_i\, y_i \;=\; x^\top y$

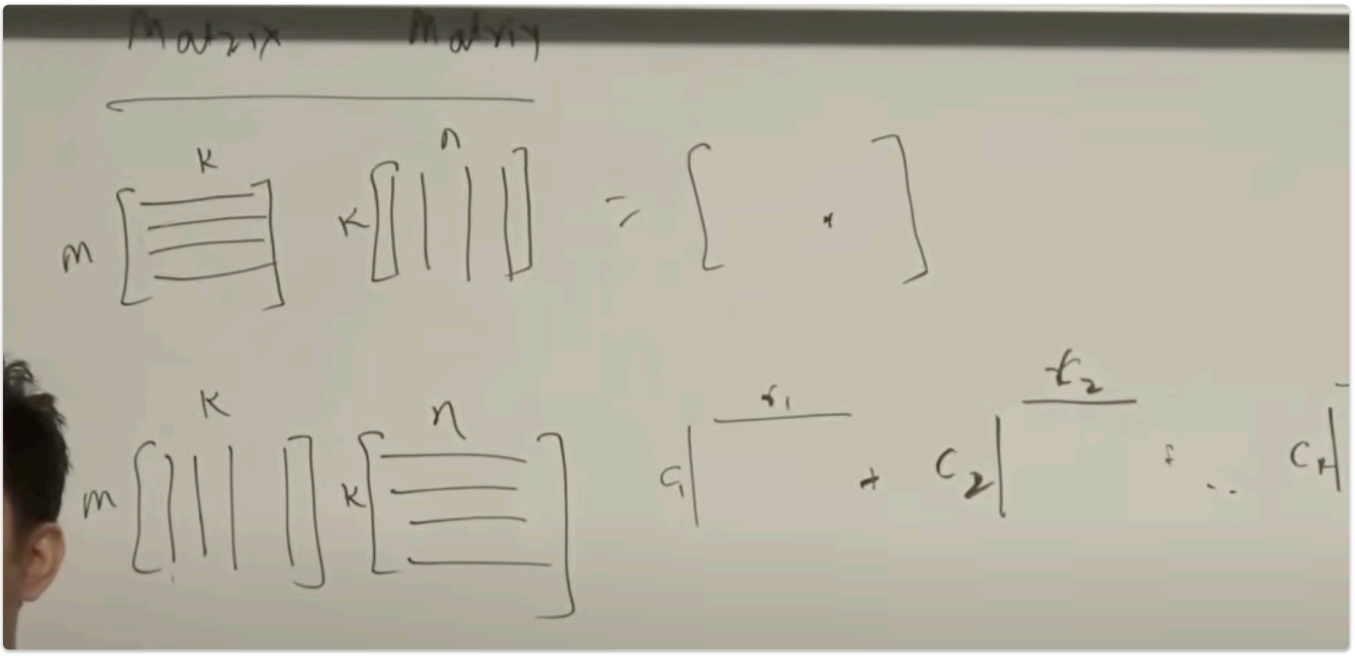
Outer product

For every $x \in \mathbb{R}^d, \quad y \in \mathbb{R}^p$, it follows $x\, y^\top, \quad y\, x^\top$

- Defined as $\sum$

Matrix-Matrix

Matrix-Matrix Multiplication



Both views start with
$A \in \mathbb{R}^{m\times k},\ B \in \mathbb{R}^{k\times n}$ and the product $C = AB \in \mathbb{R}^{m\times n}$.

Row–column (inner-product) picture

Every entry of C is the dot product of one row of A with one column of B:

$$C_{ij} = \underbrace{\begin{bmatrix} a_{i1} & a_{i2} & \dots & a_{ik} \end{bmatrix}}_{\text{row } i} \cdot \underbrace{\begin{bmatrix} b_{1j} \\ b_{2j} \\ \vdots \\ b_{kj} \end{bmatrix}}_{\text{column } j} = \sum_{r=1}^{k} a_{ir} b_{rj}.$$

Column–row (outer-product) picture

$$\begin{bmatrix} | & | & \dots & | \\ \mathbf{a}_1 & \mathbf{a}_2 & \dots & \mathbf{a}_k \\ | & | & \dots & | \end{bmatrix}_{m\times k} \begin{bmatrix} \text{---}\ \mathbf{b}_1^\top\ \text{---} \\ \text{---}\ \mathbf{b}_2^\top\ \text{---} \\ \vdots \\ \text{---}\ \mathbf{b}_k^\top\ \text{---} \end{bmatrix}_{k\times n} = \sum_{r=1}^{k} \mathbf{a}_r \mathbf{b}_r^\top.$$

→ Sum of **rank-1 matrix** additions

Why both views matter

- Inner-product view is perfect for thinking about *single entries* or implementing a naïve triple loop.
- Outer-product view exposes the product as a sum of rank-1 updates—crucial for algorithms such as **low-rank approximations** etc.

Subspaces

A **subspace** of $\mathbb{R}^d$ is any non-empty set S that

1. contains the zero vector,
2. is closed under vector addition,
3. is closed under scalar multiplication.

If those three conditions hold, S is automatically a vector space in its own right.

Imagining subspaces & understanding different dimensionalities

E.g. *Why has the subspace of the set of vectors that are orthogonal to a generic vector of R_n the dimensionality n - 1? How can I generally determine the dimensions of subspaces based on their relation to other objects?*

The general principle that holds up is that the **imposed constraints** are relevant. When working with a vector, and wanting to find orthogonal sets of vectors, we have exactly one constraint (`z^T x = 0`), which removes exactly one degree of freedom from our system. This leaves us with `n-1` (a hyperplane!).

But this also holds up with other shapes and other constraints! This notion is very **powerful!**

The four fundamental Matrix Subspaces

The **rowspace** is made out the linear combinations of the rows of a matrix A, and the **columnspace** similarly out of the linear combinations of the columns of a matrix A

The **nullspace** is a *subspace of the input space*, in which the images are zero-vector, so the input maps to *null*. The dimension is always is equal to the difference between input space & rank (in this case $3 - 2 = 1$) (see Rank-Nullity Theorem).

Relation Rowspace ↔ Nullspace

It is naturally **orthogonal to the rowspace** as it's images map always to zero (no component that is part of the rowspace). As long as any input vector has a part living in the nullspace, it will lose that specific direction.

Where do subspaces of a matrix lie?

Subspaces in a matrix ALWAYS go through the origin, as you can just multiply it by the null vector, and will always get the origin!

Symbol	Living in	"Questions it answers"	Dimension	
Column space	$\mathcal{C}(X)$	$\mathbb{R}^n$	All possible **outputs** $X\beta$	**rank** r
Row space	$\mathcal{R}(X) = \mathcal{C}(X^\top)$	$\mathbb{R}^d$	Linear combos of training **features**.	r

Symbol	Living in	"Questions it answers"	Dimension	
Null-space (kernel)	$\mathcal{N}(X)$	$\mathbb{R}^d$	Directions ζ with **zero effect** on outputs: $X\zeta = 0$.	$d - r$

> **How does input relate to the diffent matrix subspaces?**
>
> Any point in the input space, also outside of the subspace, can be decomposed into a **projection in the rowspace,** and another part, the "residual", that lies in the **nullspace.**
>
> Relevant is the *projection* on the **rowspace** as that will be mapped to the columnspace, whereas the nullspace will fall away as `An = 0`

Rank

The rank intuitively describes how many **independent directions (information)** a matrix keeps. It is the amount of "information channels" a matrix can transmit.

It also equals the **amount of non-zero eigenvalues** of a matrix. This makes sense as the eigenvectors are the different, independent directions but if an eigenvalue is zero, that reduces one dimension and also removes a direction.

Rank-Nullity Theorem

$$\text{rank}(A) + \text{nullity}(A) = n$$

The number of dimensions in the **input space** (`n`) is perfectly split between the *dimensions that are preserved* (and mapped to the output space; equal to the rank R) and the *dimensions that are lost* by being squashed to zero (the nullspace N).

Matrix Ranks

Full-rank

→ Can be always inversed! (No information loss!)

Rank-deficient

A **rank-deficient** matrix (e.g. $A : \mathbb{R}^3 \to \mathbb{R}^3, \text{rank}(A) = 2$) maps from a subspace in the *input space (→ ***rowspace****), *to another subspace in the* output space (→ ***columnspace)***

We call such a matrix also **singular**.

Eigenvectors & Eigenvalues

Determinant

→ The product of all **eigenvalues**. It tells us also whether a matrix is singular, so has a non-null Nullspace.

> **Transforming areas/volumes with matrices**
>
> If we work with a volume, the determinant also tells us the ratio of the output & input shape
> → How much does the area get scaled?

Eigenvalues

The eigenvalues of a matrix tell us **how much** the input space gets stretched in its *principal directions* (its eigenvectors)

Connection between Eigenvalues/-vectors & Nullspace

The amount of non-zero eigenvalues of a matrix always equals its rank. Inversely, it also has as many zero eigenvalues as squashed dimensions, that map to the Nullspace.

The eigenvectors corresponding to the null/non-null eigenvalues span respectively the columnspace or nullspace

Symmetric Matrices

Symmetric matrices always have real eigenvalues! That's why we love them :)

How do we calculate Eigenvectors & Eigenvalues?

We can start with the core definition of eigenvectors & eigenvalues:

$$Ax = \lambda x$$

Based on this, we can rewrite this to:

$$(A - \lambda I)x = 0$$

Symmetric Matrices

Symmetric matrices are the most important matrices in Machine Learning and make everything cleaner.

They have the property that it equals its own transpose: $A = A^T$

1. **Real Eigenvalues**: Their eigenvalues are always real numbers
2. **Orthogonal Eigenvectors**: Their eigenvectors are all perpendicular to each other - this means that there is only a stretch/shrink along this set of eigenvectors
3. **The Spectral Theorem**: This property means they can be decomposed as $A = U\Lambda U^T$, where U is an orthonormal matrix (a pure rotation) and Λ is a diagonal matrix of eigenvalues (a pure scaling).

Spectral Theorem

The **spectral theorem** tells us that for every symmetric matrix A, we can say that they have real-valued eigenvalues & orthonormal eigenvectors.

$$A = U\Lambda U^T$$

This can be read as:

1. **Rotate (`Uᵀ`):** Change from the standard basis to the basis of eigenvectors.
2. **Scale (`Λ`):** Perform a simple stretch/squeeze along these new axes.
3. **Rotate back (`U`):** Change back to the standard basis.

We can represent any symmetric matrix A as a transformation that is a **pure stretch, squeeze or flip** along a set of orthogonal principal directions!

Why no rotation/twist/shear?

Symmetric matrices cannot perform a **twist or shear** (violates orthogonal eigenvectors), or any **rotation**(which is included in these)

It still *does* change the direction of vectors but that due to the change of length in different components.

Quadratic Form

The **quadratic form** is a fundamental tool for understanding the "character" of a square matrix. It takes a matrix `A` and a vector `x` and returns a single scalar number.

- **Definition:** For a matrix $A \in \mathbb{R}^{n \times n}$ and a vector $\mathbf{x} \in \mathbb{R}^n$, the quadratic form is the function:

$$f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x}$$

> **Intuition - Input-Output Alignment**
> The expression can be seen as a dot-product between the input vector x and the output vector Ax
>
> **Hence:** A positive value tells us that the input & post-transformed input lie in the same direction (< 90 degrees to each other), and a negative value tells us they point in the opposite direction (> 90 degrees)

Definiteness

Definiteness uses the sign of the quadratic form to classify symmetric matrices. It tells us how a matrix is stretching space (into positive or negative)

- **Positive Definite (PD):** $\mathbf{x}^T A \mathbf{x} > 0$ for all non-zero $\mathbf{x}$.
  - This matrix always pushes vectors in a direction that is less than 90 degrees away from their original direction - strictly in to a positive
    - This is the case for covariance matrices and the Hessians of strictly convex functions.
- **Positive Semidefinite (PSD):** $\mathbf{x}^T A \mathbf{x} \geq 0$ for all $\mathbf{x}$.
  - This matrix pushes vectors "outward" or, in some directions, might completely squash them to the zero vector (but it never flips them).
- **Negative Definite (ND):** $\mathbf{x}^T A \mathbf{x} < 0$ for all non-zero $\mathbf{x}$.
  - *Intuition:* This matrix always flips vectors to point in a direction more than 90 degrees away from their original direction.
- **Negative Semidefinite (NSD):** $\mathbf{x}^T A \mathbf{x} \leq 0$ for all $\mathbf{x}$.
- **Indefinite:** The sign of $\mathbf{x}^T A \mathbf{x}$ depends on the direction of $\mathbf{x}$.
  - This matrix acts like a "saddle." It pushes vectors in some directions outward and flips vectors in other directions inward.

Definiteness	Condition on Eigenvalues (λ_i)	Invertible?
Positive Definite	All ($\lambda_i > 0$)	**Yes**
Positive Semidefinite	All ($\lambda_i \geq 0$)	Maybe
Negative Definite	All ($\lambda_i < 0$)	**Yes**
Negative Semidefinite	All ($\lambda_i \leq 0$)	Maybe
Indefinite	Mix of ($\lambda_i > 0$) and ($\lambda_i < 0$)	Maybe

- **The Eigenvalue Connection:** The eigenvalues represent the scaling factors along the matrix's principal axes.
  - If all eigenvalues are positive, any vector you transform will have a positive projection back onto itself.
  - If some are positive and some are negative, the result depends on which axes your input vector aligns with more.
- **The Invertibility Connection:** A matrix is invertible if and only if it has **no zero eigenvalues**.

- This means that **Positive Definite** and **Negative Definite** matrices are *always* invertible, as all their eigenvalues are strictly non-zero.